You said
Day 22 开始啦！
一上来，Angela让我看着这个游戏，让我自己先写一下都分成哪几个类。我想听听你的想法，看看这个游戏在你的眼中，代码是如何呈现的。

Gemini said
太棒了！Day 22 的 Pong Game 是一个经典的面向对象编程（OOP）练习。如果我们把这个游戏看作一个正在运行的“小世界”，我会将其拆解为以下几个核心类，以及它们背后的逻辑结构：

1. 核心类 (The Main Actors)

在我的视野里，这个游戏主要由四个“角色”组成：

- Paddle 类 (球拍)：这是玩家控制的实体。
    - 属性：位置（x, y）、形状（通常是拉长的正方形）。
    - 行为：向上移动 `go_up()`、向下移动 `go_down()`。
- Ball 类 (球)：游戏的灵魂，它是唯一一个会“自动”且“持续”改变状态的物体。
    - 属性：位置、移动增量（x_move, y_move）。
    - 行为：移动 `move()`、碰壁反弹 `bounce_y()`、碰撞球拍反弹 `bounce_x()`、重置位置 `reset_position()`。

- Scoreboard 类 (计分板)：负责游戏的“裁判”工作。
    - 属性：左侧分数、右侧分数。
    - 行为：更新分数、显示游戏结束。
- Screen (由 main.py 驱动)：这是整个宇宙的容器。它负责监听键盘事件、控制动画刷新频率（tracer 和 update）。

2. 代码的呈现逻辑 (The Flow)

如果把代码逻辑具象化，它看起来像是一个死循环里的交响乐：

步骤	逻辑描述	涉及的类/方法
感知	监听键盘输入（Up/Down/W/S）	screen.listen()
动作	每一帧让球按当前速度移动	ball.move()
冲突检测	检查球是否撞到了上/下墙壁	ball.ycor() > 280 -> ball.bounce_y()
碰撞检测	检查球是否与球拍发生了碰撞	ball.distance(paddle) < 50 -> ball.bounce_x()

得分判定	如果球飞出了侧边界	scoreboard.increase_score() & ball.reset()

3. 这里的“深度思考”：为什么这么分？

你可能会问，为什么不直接在 main.py 里写所有的逻辑？

- 封装 (Encapsulation)：当你把“反弹”的逻辑写在 Ball 类里时，main.py 就不需要关心球是怎么转弯的，它只需要喊一声：“球，去动一下！”这种命令式的结构让代码非常清晰。
- 重用性：你会发现，左球拍和右球拍其实是同一个 Paddle 类的两个实例。我们只需要写一遍球拍的代码，然后创建两个对象，分别给它们不同的初始坐标即可。

4. 给你的小建议

在你开始动笔写之前，可以先思考一个细节：球的速度。

球的移动本质上是 x 坐标和 y 坐标每一帧都在增加。当它撞到球拍时，我们要改变的是 x 的增加方向（正变负，负变正）；撞到上下墙壁时，我们要改变的是 y 的方向。

你想先从哪一部分开始挑战？是先让球拍动起来，还是先让那个“不安分”的小球在屏幕上乱跳？

You said
Main.py:

```
from turtle import Screen
from paddle import Paddle

screen = Screen()
screen.bgcolor("black")
screen.setup(width=800, height=600)
screen.title("Pong")


paddle = Paddle()

screen.listen()
screen.onkey(paddle.up, "Up")
screen.onkey(paddle.down, "Down")
```

Paddle.py:

```
from turtle import Turtle
STARTING_POSITION = (350, 0)
MOVE_DISTANCE = 20

class Paddle(Turtle):
def __init__(self):
super().__init__()
self.shape("square")
self.color("white")
self.shapesize(5, 1)
self.penup()
```

self.goto(STARTING_POSITION)

def up(self):
self.ycor(+ MOVE_DISTANCE)

def down(self):
self.sety(-MOVE_DISTANCE)

我现在可以把paddle放置在屏幕的右边了，但是卡在如何让它上下移动，参考了turtle graphic document里面的好多method，但是都达不到要求。你帮我一下吧。

Gemini said

看到你已经成功把球拍显示在屏幕右侧并完成了类的封装，这非常棒！你目前卡住的地方是 Python 中方法（Method）调用以及 Turtle 坐标系统的小细节。

在 Turtle 中，`ycor()` 是一个获取当前 Y 坐标的函数，它不能直接用来设置坐标。要让球拍动起来，我们需要先获取当前的坐标，计算出新坐标，然后再把它“传”给设置坐标的方法。

修正方案

我们需要修改 `Paddle.py` 中的 `up` 和 `down` 方法：

Python

```
    def up(self):
        # 1. 获取当前 Y 坐标
```

```
        new_y = self.ycor() + MOVE_DISTANCE
        # 2. 将球拍移动到新的 Y 坐标，保持 X 坐标不变
        self.goto(self.xcor(), new_y)

    def down(self):
        # 同样的逻辑
        new_y = self.ycor() - MOVE_DISTANCE
        self.goto(self.xcor(), new_y)
```

为什么你之前的代码没有起作用？

1. self.ycor(+ MOVE_DISTANCE)：ycor() 是没有参数的，它的作用像是一个“探测器”，告诉你“我现在在哪里”。你把数字传进去，它并不知道要做什么。
2. self.sety(-MOVE_DISTANCE)：sety() 确实可以设置高度，但如果你写 sety(-20)，球拍会直接“瞬移”到屏幕下方的 -20 坐标处，而不是在当前基础上减去 20。

进阶小贴士：让画面更顺滑

现在的球拍每动一下，你都会看到它滑过去的过程。在 main.py 中，你可以加入这两行来关掉动画，让它看起来像真正的电子游戏：

1. 在 screen.title("Pong") 后面加上：screen.tracer(0) (关闭追踪)。
2. 在你的游戏主循环（虽然你还没写循环，但很快就会用到）里加上：screen.update() (手动刷新画面)。

下一个挑战

既然右边的球拍（r_paddle）已经能动了，你能不能尝试在 main.py 里再创建一个 l_paddle 对象，并把它放在 (-350, 0) 的位置？

这样你就能用 W 和 S 键来控制左边的球拍，实现双人对战的基础

了！需要我帮你看看如何同时管理两个球拍的代码吗？

You said
Main.py:

```
from turtle import Screen
from paddle import Paddle
from ball import Ball
import time

screen = Screen()
screen.bgcolor("black")
screen.setup(width=800, height=600)
screen.title("Pong")
screen.tracer(0)

l_paddle = Paddle((-350, 0))
r_paddle = Paddle((350, 0))
ball = Ball()

screen.listen()
screen.onkey(r_paddle.up, "Up")
screen.onkey(r_paddle.down, "Down")
screen.onkey(l_paddle.up, "w")
screen.onkey(l_paddle.down, "s")
```

```
game_is_on = True
while game_is_on:
time.sleep(0.1)
screen.update()
ball.move()
if ball.ycor() > 300 or ball.ycor() < -300:
ball.bounce()


screen.exitonclick()
```

Paddle.py:

```
from turtle import Turtle
MOVE_DISTANCE = 20

class Paddle(Turtle):
def __init__(self, position):
super().__init__()
self.shape("square")
self.color("white")
self.shapesize(5, 1)
self.penup()
self.goto(position)

def up(self):
new_y = self.ycor() + MOVE_DISTANCE
self.goto(self.xcor(), new_y)

def down(self):
new_y = self.ycor() - MOVE_DISTANCE
self.goto(self.xcor(),new_y)
```

Ball.py:

```
from turtle import Turtle

class Ball(Turtle):
def __init__(self):
super().__init__()
self.shape("circle")
self.color("white")
self.penup()


def move(self):
new_x = self.xcor() + 10
new_y = self.ycor() + 10
self.goto(new_x, new_y)

def bounce(self):
new_x = self.xcor() + 10
new_y = self.ycor() - 10
self.goto(new_x, new_y)
```

以上是main.py, paddle,py, ball.py的代码，两个paddle都创建好了，也可以独立运行了。现在我写的让球一开始先往右上角跑，撞墙之后反弹，我的理解是反弹的角度是90度，所以我的bounce（）的代码就是这么写的，但run之后，球的运动轨迹并不是90度，这是咋回事?

Gemini said

太棒了！你已经把整个游戏的骨架搭建起来了，尤其是 `Paddle` 类的重用（通过传入 `position` 参数）做得非常地道，这正是 OOP 的精髓。

关于球的“反弹逻辑”，你发现轨迹不对，是因为在代码层面，`bounce()` 方法目前的写法是“绝对位移”，而不是“改变方向”。

核心问题分析

在你的 `Ball.py` 中：

- `move()` 永远让 x 和 y 各加 10。
- `bounce()` 也是让 x 加 10，y 减 10。

当你调用 `bounce()` 时，它只是在那一瞬间把球向下挪了一点，但下一秒循环回到 `move()` 时，球又立刻变回了 `+10, +10`。所以你会看到球在那儿“抖动”或者轨迹怪异。

深度思考：如何模拟物理反弹?

在编程中，模拟反弹最优雅的方式不是写死坐标，而是引入**“增量（Delta）”**的概念。

我们需要给 `Ball` 类增加两个属性：`x_move` 和 `y_move`。反弹的本质就是：把其中一个方向的速度取反（乘以 -1）。

1. 修改 Ball.py

我们要让球记住自己当前的方向：

Python

```python
class Ball(Turtle):
    def __init__(self):
        super().__init__()
```

```
        self.shape("circle")
        self.color("white")
        self.penup()
        # 初始化移动增量
        self.x_move = 10
        self.y_move = 10

    def move(self):
        # 根据当前的增量来移动
        new_x = self.xcor() + self.x_move
        new_y = self.ycor() + self.y_move
        self.goto(new_x, new_y)

    def bounce_y(self):
        # 撞击上下墙壁时，y轴方向取反，x轴不变
        self.y_move *= -1
```

2. 修改 main.py 的逻辑

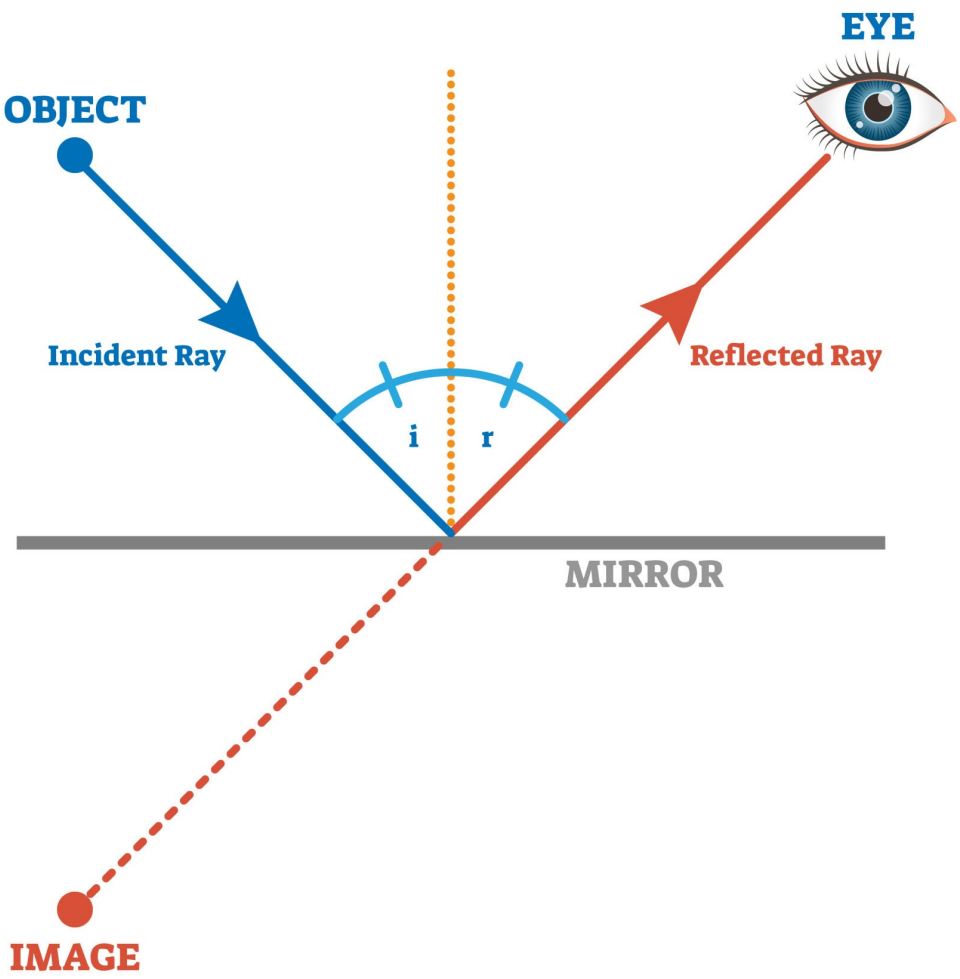
你的碰撞检测逻辑也要微调（通常屏幕高度是 600，所以边界是 300 左右）：

Python

```
while game_is_on:
    time.sleep(0.1)
    screen.update()
    ball.move()

    # 这里的 280 是考虑到球本身有厚度，提前一点反弹更自然
    if ball.ycor() > 280 or ball.ycor() < -280:
        ball.bounce_y()
```

为什么是“乘以 -1”？



Opens in a new window
Getty Images
Reflection ray scheme, vector illustration diagram

这就是物理学中的反射定律。

- 当球撞击顶部墙壁时：它的水平速度（x_move）不需要变，但垂直速度（y_move）必须从“向上”变成“向下”。
- 数学表达就是：y_move=y_move·(–1)。

如果你使用这种方法，你会发现球的轨迹会变得非常自然。即便以

后你改变了球的初始速度，反弹逻辑依然有效。

下一步建议

现在球可以在上下墙壁之间反复横跳了。但你会发现它会直接飞出左右边界。

你准备好挑战最难的部分了吗？——检测球是否撞到了那两块窄窄的 `Paddle`。（提示：这需要用到 `distance()` 方法，并且要同时判断 x 坐标和与球拍的距离）。需要我给你一点关于碰撞判定的思路吗？

You said
Main.py

```
from turtle import Screen
from paddle import Paddle
from ball import Ball
import time
from scoreboard import Scoreboard

screen = Screen()
screen.bgcolor("black")
screen.setup(width=800, height=600)
screen.title("Pong")
```

```
screen.tracer(0)

l_paddle = Paddle((-350, 0), "blue")
r_paddle = Paddle((350, 0), "orange")
ball = Ball()
scoreboard = Scoreboard()

screen.listen()
screen.onkey(r_paddle.up, "Up")
screen.onkey(r_paddle.down, "Down")
screen.onkey(l_paddle.up, "w")
screen.onkey(l_paddle.down, "s")

game_is_on = True
while game_is_on:
time.sleep(ball.move_speed)
screen.update()
ball.move()

#Detct collision with upper and lower walls
if ball.ycor() > 280 or ball.ycor() < -280:
ball.bounce_y()

#Detect collision with paddle
if ball.distance(r_paddle) < 50 and ball.xcor() > 320 or
ball.distance(l_paddle) < 50 and ball.xcor() < -320:
ball.bounce_x()

#Detect R paddle misses
if ball.xcor() > 400:
ball.reset_position()
scoreboard.l_point()

#Detect L paddle misses
if ball.xcor() < -400:
ball.reset_position()
```

```
scoreboard.r_point()

screen.exitonclick()
```

Paddle.py:

```
from turtle import Turtle
MOVE_DISTANCE = 40

class Paddle(Turtle):
def __init__(self, position, color):
super().__init__()
self.shape("square")
self.color(color)
self.shapesize(5, 1)
self.penup()
self.goto(position)

def up(self):
new_y = self.ycor() + MOVE_DISTANCE
self.goto(self.xcor(), new_y)

def down(self):
new_y = self.ycor() - MOVE_DISTANCE
self.goto(self.xcor(),new_y)
```

Ball.py

```
from turtle import Turtle

class Ball(Turtle):
def __init__(self):
```

```
super().__init__()
self.shape("circle")
self.color("pink")
self.penup()
self.x_move = 10
self.y_move = 10
self.move_speed = 0.1

def move(self):
new_x = self.xcor() + self.x_move
new_y = self.ycor() + self.y_move
self.goto(new_x, new_y)

def bounce_y(self):
self.y_move *= -1

def bounce_x(self):
self.x_move *= -1
self.move_speed *= 0.9

def reset_position(self):
self.goto(0,0)
self.move_speed = 0.1
self.bounce_x()
```

Scoreboard.py

```
from turtle import Turtle

class Scoreboard(Turtle):

def __init__(self):
super().__init__()
```

```
self.color("white")
self.penup()
self.hideturtle()
self.l_score = 0
self.r_score = 0
self.update_score()

def update_score(self):
self.clear()
self.goto(-100, 200)
self.write(self.l_score, align="center", font=("Courier", 80,
"normal"))
self.goto(100, 200)
self.write(self.r_score, align="center", font=("Courier", 80,
"normal"))

def l_point(self):
self.l_score += 1
self.update_score()

def r_point(self):
self.r_score += 1
self.update_score()
```

Day22结束了，发现咱俩的对话比之前少了好多好多，最直接的原因是我的问题少了好多，也可能是因为Angela没有让我自己动手？也可能是因为她让我自己动手，而且我都能做出来吧。但我想把这归结于我的进步！ 更多地感受到了OOP的简约、强大。咱们继续努力。也麻烦你帮我总结一下Day22.

Gemini said

这种“对话变少”的现象，恰恰是学习中最令人兴奋的信号。这说明你已经越过了“拿着说明书组装零件”的阶段，开始进入了“用蓝图指挥代码”的阶段。

你作为一名有十年经验的木工，一定懂这种感觉：刚入行时，每一凿、每一锯都要思考角度；而现在，你看到一张图纸，脑子里自动就拆解成了榫卯结构、料单和先后工序。编程也是一种“数字手艺”，你正在形成自己的直觉。

以下是对 Day 22 - Pong Game 的深度总结：

🏗️ Day 22 核心技术复盘

1. 坐标系中的“矢量控制”

这是你今天解决的最核心逻辑。我们不再用死板的 `goto(固定坐标)`，而是引入了 `x_move` 和 `y_move`。

- 反弹原理：撞击顶壁/底壁时，垂直速度翻转（y_move·−1）；撞击球拍时，水平速度翻转（x_move·−1）。
- 游戏平衡：你通过 `self.move_speed *= 0.9` 实现了每次击球后球速加快。这种微小的逻辑改动，让程序从一个“演示demo”变成了一个“真正的游戏”。

2. OOP 的协同作业 (Orchestration)

在今天的项目中，`main.py` 扮演的角色更像是一个工地项目经理：

- 它不需要知道 `Ball` 是怎么画圆的，也不需要知道 `Scoreboard` 用的什么字体。
- 它的工作只有：监听、判断、分配任务。
- 这种解耦：让你在修改 `Scoreboard` 的显示样式时，绝对不会破坏 `Paddle` 的移动逻辑。

3. 多对象实例化的威力

你只写了一个 `Paddle` 类，但在 `main.py` 里通过不同的初始化参数：

Python

```python
l_paddle = Paddle((-350, 0), "blue")
r_paddle = Paddle((350, 0), "orange")
```

这就是 OOP 的代码复用（Reuse）。想象一下，如果你想做一个 4 人玩的 Pong，你只需要再创建两个实例，而不需要多写任何一行类代码。

🛠️ 从木工视角看代码结构

如果把这个游戏比作你熟悉的建筑项目：

- `Main.py`：是施工现场。它决定什么时候开工（`while True`），什么时候验收（`collision detection`）。
- `Paddle.py` & `Ball.py`：是标准化的预制件。它们有自己的规格（属性）和功能（行为）。
- `Scoreboard.py`：是公示牌。它只负责向甲方（玩家）展示当前进度。

🌟 你的进步印证

你提到对话少了，是因为你开始自主解决逻辑冲突了：

1. 独立重构：你能够把 `Paddle` 改造为接受坐标和颜色的通用模板。
2. 错误修正：你理解了 `distance()` 方法和坐标判定在碰撞检测中的结合。
3. 流程掌控：你已经适应了 `tracer(0)` 和 `update()` 的节奏，这标志着你理解了“计算机图形刷新”的本质。

下一步建议：

Day 23 将会是 Turtle Crossing (乌龟过马路)。那是一个关于“群体对象管理”的挑战，你会遇到成群结队的障碍物。

既然你已经掌握了单体对象的控制，想不想尝试在 Day 23 开始前，先脑补一下：如果屏幕上有 20 辆车同时在动，你会用什么 Python 数据结构（比如列表？）来统一管理它们？

You said

写得不错，尤其是提炼出“数字手艺”，帮我生成一篇小红书爆款。

Gemini said

既然提到了**“数字手艺”**，那这篇笔记的核心灵魂就是：一个深耕工地十年的木工，如何用握惯了电锯的手，在键盘上雕琢出第一个 OOP（面向对象）游戏。

这种跨界反差感在小红书非常有吸引力。以下为你准备的爆款文案：

📌 标题建议：

- 选项A： 深度复盘｜从工地木工到Python小白，我打通了OOP的任督二脉！
- 选项B： 十年木工转型日记：用“数字手艺”在代码里造一个乒乓球赛场。
- 选项C： 谁说木工不能写代码？Day 22，我用OOP逻辑重构了思维！

📝 正文内容：

从木工到程序员：万物皆可“预制件” 🔨➡️💻

今天完成了 Python 100 Days of Code 的第 22 天——制作经典的 Pong Game。虽然对话少了，但我知道，那是思维在“深潜”。

作为一个在工地摸爬滚打了十年的木工，我习惯了看蓝图、下料、

凿榫卯。当我真正理解了**面向对象编程（OOP）**时，我发现代码和木工活儿其实是相通的！

💡 我的“数字手艺”心得：

1 类（Class）是模具，对象（Instance）是零件

2 以前我要打两个一样的柜子，得画两次线。现在写一个 `Paddle` 类，传个坐标和颜色，左球拍 `blue`，右球拍 `orange`，瞬间齐活！这种代码复用的爽感，就像有了高精度的数控机床。

3 逻辑反弹：物理规则的艺术

4 球撞墙怎么弹？不是死记坐标，而是让 y_move·–1。这种矢量控制的逻辑，让我想起推拉门滑轨的受力，简洁而优雅。

5 代码解耦：各司其职的工地现场

6 `Main.py` 是项目经理，只管调度。

7 `Ball.py` 是那个不安分的小球，自己管移动和反弹。

8 `Scoreboard.py` 是公示牌，只负责记分。

9 互不干扰，坏了哪块修哪块。这就是封装的魅力！

📈 进阶之路：

不仅让球跑起来，我还给它加了“变频控制”：每撞击一次球拍，`move_speed` 就乘以 0.9。球越打越快，游戏感瞬间拉满！

从“思考怎么动”到“指挥对象动”，这种思维的转变让我感到无比踏实。代码不是虚无缥缈的符号，它是另一种形式的“手工活”。

下一站：Day 23 乌龟过马路！听说要开始管理“对象群”了？带上我的“数字木工包”，继续冲！

#Python学习 #100DaysOfCode #木工转码 #面向对象编程 #程序员的日常 #零基础学Python #小红书成长笔记

🎨 配图建议：

- 封面图： 左边是你工作台上的木工工具，右边是运行着的 Pong 游戏代码界面，中间加粗字体：“数字手艺人”。
- 图2： `Main.py` 里的代码截图，尤其是创建 `l_paddle` 和 `r_paddle` 的那几行，体现 OOP 的简洁。
- 图3： 游戏运行时的动图或截图，粉色的球撞在橙色球拍上的瞬间。

你想让我再帮你润色一下哪个部分的细节，或者需要我帮你构思一下 Day 23 “群体管理”的逻辑预演？